ICT912 Programming

Topic 10
File Operations

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 9:

- Explain the role of inheritance in building a class hierarchy
- Explore the types of inheritance and necessary components
- Explore how polymorphism is used in OOP
- Dealing with Exceptions
- Using Exceptions

Week 10: Lesson Learning Outcomes

- What is file handling?
- Explain the use of I/O functions
- Explain file processing operations
- Understand how to read, close or create a new file
- Define `os` module

File Input and Output (I/O)

- **File input and output, or file I/O**
  - **File input:** code reads information from a file
  - **File output:** code writes information to a file

- Most programming languages have a **file handler**, a special built-in feature that allows you to interact with a file
  - Allows you to request information about a file
  - Provides features to enable I/O, opening files, and closing files

Streaming and Buffering

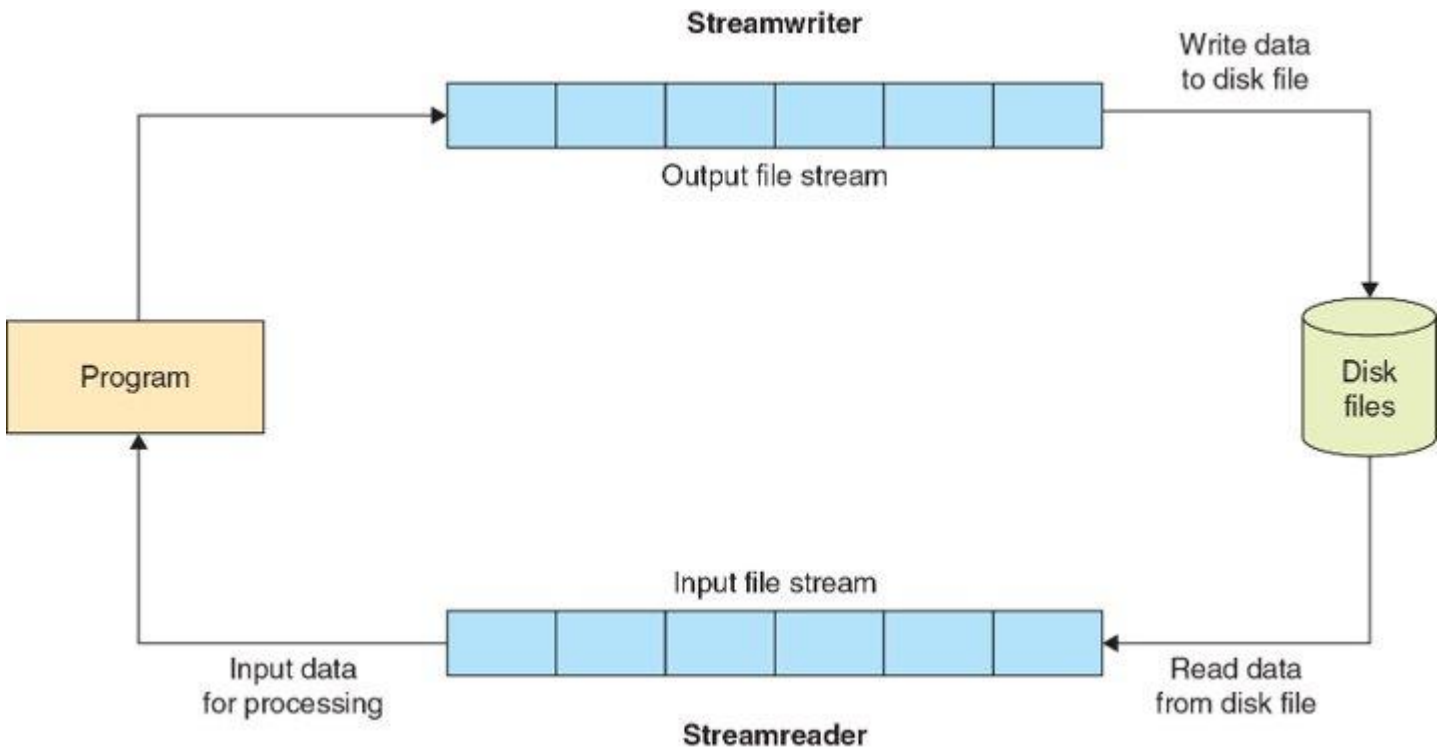
- **Streaming and buffering**
- **Stream:** a channel that allows you to interact with files stored on a computer or another drive

- Some programming languages use a **streamwriter** to create a stream for writing text data to a file and a **streamreader** to create a stream for reading text from a file

- Computers use **buffering** to move data, **bit by bit**, into and out of a reserved part of memory (called a buffer) that code can access

- Using a **buffered stream** can improve performance when the program must read and write large amounts of data

Streaming and buffering



McMullen/Matthews/Parsons, Programming with Python, 1st Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Reading from a Text File

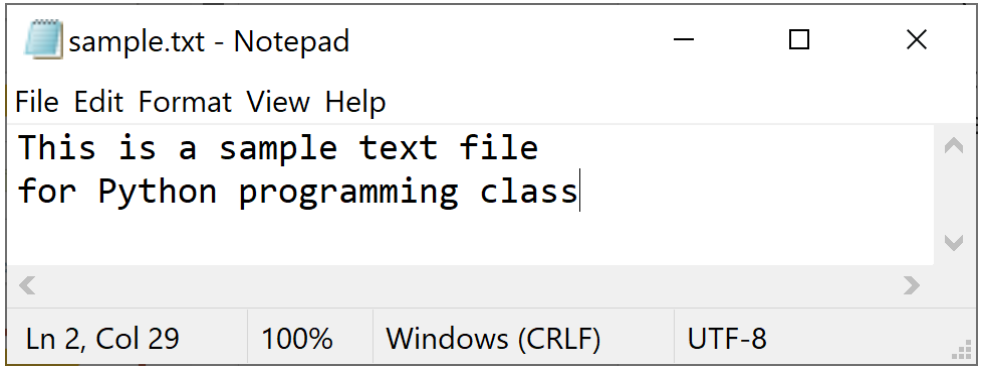
```
file_variable = open("songs.txt", access_mode)
```

- Read mode ("r"): opens the specified file so contents can be retrieved; default

```
1  file_handler = open("sample.txt","r")
2
3  print(file_handler.read())
```

```
This is a sample text file
for Python programming class

Process finished with exit code 0
```



Closing Files After Use

- After a file is closed, it becomes available to be opened by other code
- Syntax: `file_variable.close()`
- The `with` and `as` keywords create a temporary file variable and close it immediately at the end of the code block—the calls to `open`() and `close`() are done for you

Creating and Writing New Files

- Creating a file
  - To create a new file, you assign a file what is returned by open(), but with "w" as the access mode.

```
output_file = open("favorite_songs.txt", "w")
```

- Opening a file for writing
  - Use the "w" access mode to open a file for writing (overwriting any existing contents)
  - Use the "a" access mode to add content to the end of a file that already contains data

Creating and Writing New Files

```python
# open the file1.txt in write mode
file1 = open("file.txt","w")

# write contents to the file1.txt file
file1.write("Programming is Fun.\n")
file1.write("This is Python programming class.\n")

# open a file in read mode
file1 = open("file.txt")

# read the file content
read_file = file1.read()
print(read_file)
```

```python
# open the file1.txt in append mode
file1 = open("file.txt","a")

# write contents to the file1.txt file
file1.write("Python Programming is easy.\n")

# open a file in read mode
file1 = open("file.txt")

# read the file content
read_file = file1.read()
print(read_file)
```

```
Programming is Fun.
This is Python programming class.

Programming is Fun.
This is Python programming class.
Python Programming is easy.
```

The os Module

- os (operating system) module for import that contains many useful file-related functions

- The `os.path.join()` function, for example, combines an iterative sequence of strings into a file path using the appropriate separator depending on the operating system at runtime

- The `os.path.listdir()` function generates a list of strings of the files in a directory whose path is supplied as its argument

The os Module

Function	Description	Code Example
os.path.join()	Joins any number of strings into a path using the appropriate separator based on your current OS	print(os.path.join("folder", "file.txt"))
os.path.isfile()	Given a path, returns True if the path points to a file (not a folder), False otherwise	os.path.isfile("songs.txt")
os.path.isdir()	Given a path, returns True if the path points to a directory (folder), False otherwise	os.path.isdir("top_50")
os.path.exists()	Given a path, returns True if the path points to a file or directory, False otherwise	os.path.exists ("folder\song.txt")
os.listdir()	Given a path to a directory, returns a list containing strings representing all files and directories within	os.listdir("poems")
os.remove()	Given a path to a file, removes the file; does not work with directories or folders	os.remove("songs.txt")
os.rmdir()	Given a path to a directory, removes the directory; does not work with files or nonempty directories	os.remove("poems")

Examples

```python
import os

folder = input("Enter a folder name: ")
filename = input("Enter a filename: ")

filepath = os.path.join(folder, filename)

print(filepath)
```

OUTPUT:

```
Enter a folder name: top_50 [Enter]
Enter a filename: bye.mp3 [Enter]
top_50\bye.mp3
```

Using `os.path.join()` to create a file path

ex5.py | ex3.py | ex1.py | file.py | newfile.py | newos.py

```python
# importing os module
import os

# os.listdir() method return path
dir_list = os.listdir()
print("Files and directories in  current working directory :")

# print the list
print(dir_list)
```

Console: connection closed (Running: 1 seg)

```
Files and directories in  current working directory :
['.vpl_launcher.sh', 'vpl_execution', 'vpl_environment.sh', 'newos.py', 'newfile
.py', 'file.py', 'ex5.py', 'ex3.py', 'ex1.py', 'common_script.sh']
```

Listing all files in the current folder using `os.listdir()`

Example

```
import os

folder = input("Enter a folder name: ")
all_files = os.listdir(folder)
removed_files = 0

for file in all_files:
    file_path = os.path.join(folder, file)
    if os.path.isfile(file_path) and not file.endswith(".txt"):
        os.remove(file_path)
        removed_files += 1

print(removed_files, "files were removed.")
```

OUTPUT:

```
Enter a folder name: poems [Enter]
7 files were removed.
```

Removing files that do not have a `".txt"` extension

Next Week – What's Coming Up?

- Assessment 2 Presentations

CIHE

CIHE Australia